Variables y tipos

In [3]:
```
x = 10
print(x)

cadena= "Hola Mundo"
print(cadena)
```

```
10
Hola Mundo
```

In [ ]:
```
```

In [4]:
```
x = y = z = 10
print(x, y, z)
```

```
10 10 10
```

In [5]:
```
type(x)
```

Out[5]:
```
int
```

In [6]:
```
type(cadena)
```

Out[6]:
```
str
```

In [7]:
```
x = "Hola Mundo"
cadena = 10
```

In [8]:
```
type(x)
```

Out[8]:
```
str
```

In [9]:
```
type(cadena)
```

Out[9]:
```
int
```

In [10]:
```
SEGUNDOS_POR_DIA = 60 * 60 * 24
PI = 3.14
```

Cadenas

In [11]:
```
cadena1 = 'Hola '
cadena2 = "Mundo"
print(cadena1)
print(cadena2)
concat_cadenas = cadena1 + cadena2
print(concat_cadenas)
```

```
Hola
Mundo
Hola Mundo
```

In [12]:
```
num_cadena = concat_cadenas + ' ' + str(10)
print(num_cadena)
```

```
Hola Mundo 10
```

In [13]:
```
num_cadena = "{} {} {}".format(cadena1, cadena2, 10)
print(num_cadena)
```

```
Hola  Mundo 10
```

In [14]:
```
num_cadena = "Cambiando el orden {1} {2} {0} #".format(cadena1, cadena2, 10)
print(num_cadena)
```

```
Cambiando el orden Mundo 10 Hola  #
```

Operadores

In [15]:
```
print(1 + 5)
print(6*3)
print(10-4)
print(100/10)
print(10%3)
print(((20*3)+(10+1))/10)
print(2**2)
```

```
6
18
6
10.0
1
7.1
4
```

In [16]:
```
False and True
```

Out[16]: False

In [17]:
```
print(7<5)
print(7>5)
print((11*3)+2 == 36-1)
print((11*3)+2 >= 36)
print("curso" != "CurSO")
```

```
False
True
True
False
True
```

Listas

In [18]:
```
listaDiasDelMes=[31,28,31,30,31,30,31,31,30,31,30,31]

print(listaDiasDelMes)
print(listaDiasDelMes[0])
print(listaDiasDelMes[6])
print(listaDiasDelMes[11])
```

```
[31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
31
31
31
```

In [19]:
```
listaNumeros = [['cero', 0],['uno',1,'UNO'],['dos',2],['tres',3],['cuatro',4],


print(listaNumeros)
print(listaNumeros[0])
print(listaNumeros[1])

print(listaNumeros[2][0])
print(listaNumeros[2][1])

print(listaNumeros[1][0])
print(listaNumeros[1][1])
print(listaNumeros[1][2])
```

```
[['cero', 0], ['uno', 1, 'UNO'], ['dos', 2], ['tres', 3], ['cuatro', 4], ['X',
5]]
['cero', 0]
['uno', 1, 'UNO']
dos
2
uno
1
UNO
```

In [20]:
```
listaNumeros[5][0] = "cinco"
print(listaNumeros[5])
```

```
['cinco', 5]
```

Tuplas

In [21]:
```
tuplaDiasDelMes = (31,28,31,30,31,30,31,31,30,31,30,31)

print(tuplaDiasDelMes)
print(tuplaDiasDelMes[0])
print(tuplaDiasDelMes[3])
print(tuplaDiasDelMes[1])
```

```
(31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31)
31
30
28
```

In [22]:
```
tuplaNumeros = (('cero', 0),('uno',1,'UNO'),('dos',2),('tres',3),('cuatro',4),

print(tuplaNumeros)
print(tuplaNumeros[0])
print(tuplaNumeros[1])

print(tuplaNumeros[2][0])
print(tuplaNumeros[2][1])

print(tuplaNumeros[1][0])
print(tuplaNumeros[1][1])
print(tuplaNumeros[1][2])
```

```
(('cero', 0), ('uno', 1, 'UNO'), ('dos', 2), ('tres', 3), ('cuatro', 4), ('X',
5))
('cero', 0)
('uno', 1, 'UNO')
dos
2
uno
1
UNO
```

In [23]:
```
print("valor actual {}".format(listaDiasDelMes[0]))
listaDiasDelMes[0] = 50
print("valor cambiado {}".format(listaDiasDelMes[0]))
tuplaDiasDelMes[0] = 50
```

```
valor actual 31
valor cambiado 50
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[23], line 4
      2 listaDiasDelMes[0] = 50
      3 print("valor cambiado {}".format(listaDiasDelMes[0]))
----> 4 tuplaDiasDelMes[0] = 50

TypeError: 'tuple' object does not support item assignment
```

Tupla con Nombre

In [24]:
```
from collections import namedtuple

planeta = namedtuple('planeta', ['nombre', 'numero'])

planeta1 = planeta('Mercurio', 1)
print(planeta1)
```

```
planeta2 = planeta('Venus', 2)

print(planeta1.nombre, planeta1.numero)

print(planeta2[0], planeta2[1])

print('Campos de la tupla {}'.format(planeta1._fields))
```

```
planeta(nombre='Mercurio', numero=1)
Mercurio 1
Venus 2
Campos de la tupla ('nombre', 'numero')
```

Diccionario

In [25]:
```
elementos = {'hidrogeno': 1, 'helio': 2, 'carbon': 6}

print(elementos)

print(elementos['hidrogeno'])
```

```
{'hidrogeno': 1, 'helio': 2, 'carbon': 6}
1
```

In [26]:
```
elementos['litio'] = 3
elementos['hidrogeno'] = 8

print(elementos)
```

```
{'hidrogeno': 8, 'helio': 2, 'carbon': 6, 'litio': 3}
```

In [27]:
```
elementos2 = {}
elementos2['H'] = {'name': 'Hydrogen', 'number': 1, 'weight': 1.00794}
elementos2['He'] = {'name': 'Helium', 'number': 2, 'weight': 4.002602}

print(elementos2)
```

```
{'H': {'name': 'Hydrogen', 'number': 1, 'weight': 1.00794}, 'He': {'name': 'Hel
ium', 'number': 2, 'weight': 4.002602}}
```

In [28]:
```
print(elementos2['H'])
print(elementos2['H']['name'])
print(elementos2['H']['number'])

elementos2['H']['weight'] = 4.30
print(elementos2['H']['weight'])
```

```
{'name': 'Hydrogen', 'number': 1, 'weight': 1.00794}
Hydrogen
1
4.3
```

In [29]:
```
elementos2['H'].update({'gas noble': True})
```

```
elementos2['H']
```

Out[29]:
```
{'name': 'Hydrogen', 'number': 1, 'weight': 4.3, 'gas noble': True}
```

In [30]:
```
print(elementos2.items())

print(elementos2.keys())
```

```
dict_items([('H', {'name': 'Hydrogen', 'number': 1, 'weight': 4.3, 'gas noble': True}), ('He', {'name': 'Helium', 'number': 2, 'weight': 4.002602})])
dict_keys(['H', 'He'])
```

Funciones

In [31]:
```
def imprime_nombre(nombre):
    print('Hola '+nombre)
```

In [32]:
```
imprime_nombre("JJ")
```

```
Hola JJ
```

In [33]:
```
def cuadrado(x):
    return x**2
```

In [34]:
```
x = 5

print("El cuadrado de {} es {}".format(x, cuadrado(x)))
```

```
El cuadrado de 5 es 25
```

In [35]:
```
def varios(x):
    return x**2, x**3, x**4
```

In [36]:
```
val1, val2, val3 = varios(2)
```

In [37]:
```
print("{} {} {}".format(val1,val2,val3))
```

```
4 8 16
```

In [38]:
```
def cuadrado_default(x=3):
    return x**2
```

In [39]:
```
cuadrado_default()
```

Out[39]:
```
9
```

In [40]:
```
val4, _, val5 = varios(2)
print("{} {}".format(val4, val5))
```

```
4 16
```

Variables Globales

In [41]:
```
vg = 'Global'
```

In [42]:
```
def funcion_v1():
    print(vg)
```

In [43]:
```
funcion_v1()

print(vg)
```

```
Global
Global
```

In [44]:
```
def funcion_v3():
    print(vg)
    vg = 'Local'
    print(vg)
```

In [45]:
```
funcion_v3()
```

```
---------------------------------------------------------------------------
UnboundLocalError                         Traceback (most recent call last)
Cell In[45], line 1
----> 1 funcion_v3()

Cell In[44], line 2, in funcion_v3()
      1 def funcion_v3():
----> 2     print(vg)
      3     vg = 'Local'
      4     print(vg)

UnboundLocalError: cannot access local variable 'vg' where it is not associated
with a value
```

In [46]:
```
def funcion_v4():
    global vg
    print(vg)
    vg="Local"
    print(vg)
```

In [47]:
```
funcion_v4()

print(vg)
```

```
Global
Local
Local
```

Estructuras de control selectivas

In [48]:
```
def obtenerMayor(param1, param2):
    if param1 < param2:
        print('{} es mayor que {}'.format(param2, param1))
```

In [49]:
```
obtenerMayor(5,7)
```

```
7 es mayor que 5
```

In [50]:
```
obtenerMayor(7, 5)
```

In [51]:
```
x = y = z = 3
if x == y == z:
    print(True)
```

```
True
```

In [52]:
```
def obtenerMayorv2(param1, param2):
    if param1 < param2:
        return param2
    else:
        return param1
```

In [53]:
```
print("El mayor es {}".format(obtenerMayorv2(4, 20
                                              )))
```

```
El mayor es 20
```

In [54]:
```
print("El mayor es {}".format(obtenerMayorv2(11, 6)))
```

```
El mayor es 11
```

In [55]:
```
def obtenerMayor_idiom(param1, param2):
    valor = param2 if (param1 < param2) else param1
    return valor
```

In [56]:
```
print("El mayor es {}".format(obtenerMayor_idiom(11, 6)))
```

```
El mayor es 11
```

In [57]:
```
def numeros(num):
    if num == 1:
        print("Tu numero es 1")
    elif num == 2:
        print(" el numero es el 2")
    elif num == 3:
        print(" el numero es el 3")
    elif num == 4:
        print(" el numero es el 4")
    else:
        print("no hay opcion")
```

In [58]:
```
numeros(2)
```

```
el numero es el 2
```

In [59]:
```
numeros(5)
```

```
no hay opcion
```

In [60]:
```
def numeros_idiom(num):
    if num in (1,2,3,4):
        print("Tu numero es {}".format(num))
    else:
        print("{} no es una opcion.". format(num))
```

In [61]:
```
numeros_idiom(2)
```

```
Tu numero es 2
```

In [62]:
```
numeros_idiom(5)
```

```
5 no es una opcion.
```

In [63]:
```
def obtenerMasGrande(a, b, c):
    if a>b:
        if a>c:
            return a
        else:
            return c
    else:
        if b>c:
            return b
        else:
            return c
```

In [64]:
```
print("El numero mas grande es {}". format(obtenerMasGrande(7,13,1)))
```

```
El numero mas grande es 13
```

Estructuras de control repetitivas

In [65]:
```
def cuenta(limite):
    i = limite
    while True:
        print(i)
        i =i-1
        if i==0:
            break
```

In [66]:
```
cuenta(10)
```

```
10
9
8
7
6
5
4
3
2
1
```

In [67]:
```
def factorial(n):
    i=2
    tmp = 1
    while i < n+1:
        tmp = tmp * i
        i = i + 1
    return tmp
```

In [68]:
```
print(factorial(4))
```

```
24
```

In [69]:
```
print(factorial(6))
```

```
720
```

In [70]:
```
for x in [1,2,3,4,5]:
    print(x)
```

```
1
2
3
4
5
```

In [71]:
```
for x in range(5):
    print(x)
```

```
0
1
2
3
4
```

In [72]:
```
for x in range(-5, 2):
    print(x)
```

```
-5
-4
-3
-2
-1
0
1
```

In [73]:
```
for num in ["uno", "dos", "tres", "cuatro"]:
    print(num)
```

```
uno
dos
tres
cuatro
```

In [74]:
```
elementos = {'hidrogeno': 1, 'helio': 2, 'carbon': 6}

for llave, valor in elementos.items():
    print(llave, " = ", valor)
```

```
hidrogeno  =  1
helio  =  2
carbon  =  6
```

In [75]:
```
elementos = {'hidrogeno': 1, 'helio': 2, 'carbon': 6}

for llave in elementos.keys():
    print(llave)
```

```
hidrogeno
helio
carbon
```

In [76]:
```
for valor in elementos.values():
    print(valor)
```

```
1
2
6
```

In [77]:
```
for idx, x in enumerate(elementos):
    print("El indice es: {} y el elemento: {}".format(idx, x))
```

```
El indice es: 0 y el elemento: hidrogeno
El indice es: 1 y el elemento: helio
El indice es: 2 y el elemento: carbon
```

In [78]:
```
def cuenta_idiom(limite):
    for i in range(limite, 0, -1):
        print(i)
    else:
        print("Cuenta finalizada")
```

In [79]:
```
cuenta_idiom(5)
```

```
5
4
3
2
1
Cuenta finalizada
```

In [80]:
```
def cuenta_idiomv2(limite):
```

```
    for i in range(limite, 0, -1):
        print(i)
        if i == 3:
            break
    else:
        print("Cuenta finalizada")
```

In [81]:
```
cuenta_idiomv2(5)
```

```
5
4
3
```

Bibliotecas

In [82]:
```
import math

x = math.cos(math.pi)

print(x)
```

```
-1.0
```

In [83]:
```
from math import *

x = cos(pi)

print(x)
```

```
-1.0
```

In [84]:
```
from math import cos, pi

x = cos(pi)

print(x)
```

```
-1.0
```

In [85]:
```
print(dir(math))
```

```
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos', 'acos
h', 'asin', 'asinh', 'atan', 'atan2', 'atanh', 'cbrt', 'ceil', 'comb', 'copysig
n', 'cos', 'cosh', 'degrees', 'dist', 'e', 'erf', 'erfc', 'exp', 'exp2', 'expm
1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd', 'hyp
ot', 'inf', 'isclose', 'isfinite', 'isinf', 'isnan', 'isqrt', 'lcm', 'ldexp',
'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf', 'nan', 'nextafter', 'perm',
'pi', 'pow', 'prod', 'radians', 'remainder', 'sin', 'sinh', 'sqrt', 'sumprod',
'tan', 'tanh', 'tau', 'trunc', 'ulp']
```

In [86]:
```
help(math.log)
```

```
Help on built-in function log in module math:

log(...)
    log(x, [base=math.e])
    Return the logarithm of x to the given base.

    If the base is not specified, returns the natural logarithm (base e) of x.
```

In [87]:
```
import math as ma

x = ma.cos(ma.pi)

print(x)
```

```
-1.0
```

Graficacion

In [88]:
```
%pylab inline
```

```
%pylab is deprecated, use %matplotlib inline and import the required libraries.
Populating the interactive namespace from numpy and matplotlib
C:\Users\monti\AppData\Roaming\Python\Python312\site-packages\IPython\core\magics\pylab.py:162: UserWarning: pylab import has clobbered these variables: ['pi', 'atanh', 'log10', 'asin', 'trunc', 'isnan', 'pow', 'prod', 'ldexp', 'atan2', 'degrees', 'isfinite', 'lcm', 'inf', 'gamma', 'sinh', 'ma', 'acosh', 'tanh', 'radians', 'cbrt', 'modf', 'asinh', 'hypot', 'remainder', 'nan', 'tan', 'log1p', 'cos', 'log', 'isclose', 'isinf', 'sin', 'gcd', 'ceil', 'nextafter', 'fmod', 'e', 'cosh', 'expm1', 'log2', 'sqrt', 'fabs', 'acos', 'exp2', 'floor', 'frexp', 'atan', 'copysign', 'exp']
`%matplotlib` prevents importing * from pylab and numpy
  warn("pylab import has clobbered these variables: %s"  % clobbered +
```

In [89]:
```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
```

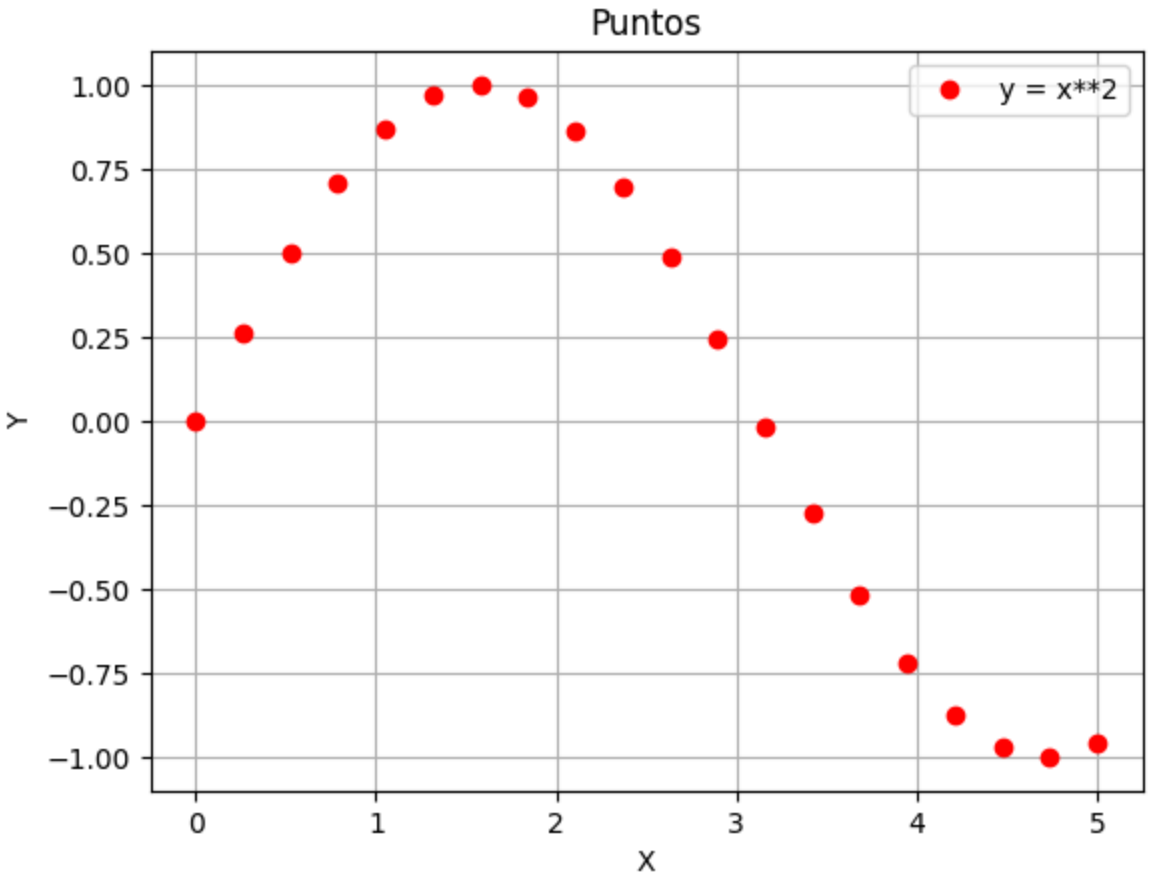
In [90]:
```
x = linspace(0, 5 ,20)
```

In [91]:
```
fig, ax = plt.subplots(facecolor='w', edgecolor='k')
ax.plot(x, sin(x), marker="o", color='r', linestyle='None')

ax.grid(True)
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.grid(True)
ax.legend(["y = x**2"])

plt.title('Puntos')
plt.show()

fig.savefig("Grafica.png")
```


Puntos
y = x**2
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
Y
0
1
2
3
4
5
X